

1. Introduction

Conditional statements are one of the fundamental concepts in programming. They allow a program to make decisions based on whether a condition is true or false.

Python provides three main keywords for conditional execution:

- `if`
- `elif`
- `else`

Python's conditional syntax is designed to be simple and readable. Unlike languages such as C++, Java, and C#, Python uses **indentation instead of braces** `{ }` to define blocks of code. Python's official documentation describes `elif` as a convenient alternative to repeatedly nesting `else if` statements.

2. The `if` Statement

The `if` statement executes a block of code when its condition evaluates to true.

Syntax

```
if condition:
    statement
```

Example

```
age = 20

if age >= 18:
    print("Adult")
```

Output:

```
Adult
```

The colon `:` indicates that a new block begins, and indentation determines which statements belong to that block.

3. The `if-else` Statement

The `else` block executes when the `if` condition is false.

```
age = 16

if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Output:

Minor

Only one of the two branches is executed.

4. The `if-elif-else` Statement

When there are multiple possible conditions, Python uses `elif`, which means "else if."

```
mark = 85

if mark >= 90:
    grade = "A"
elif mark >= 80:
    grade = "B"
elif mark >= 70:
    grade = "C"
else:
    grade = "D"

print(grade)
```

Output:

B

Python evaluates the conditions **from top to bottom**. Once one condition is true, its block is executed and the remaining `elif` and `else` blocks are skipped. Python allows zero or more `elif` clauses, while `else` is optional.

5. How Python Evaluates Conditions

A condition is an expression that Python evaluates to determine whether it should execute a block.

Conditions commonly use:

1. Comparison operators
 2. Logical operators
 3. Membership operators
 4. Identity operators
 5. Truthy and falsy values
-

5.1 Comparison Operators

Comparison operators compare two values.

Operator	Meaning	Example
==	Equal	x == 5
!=	Not equal	x != 5
>	Greater than	x > 5
<	Less than	x < 5
>=	Greater than or equal	x >= 5
<=	Less than or equal	x <= 5

Example:

```
temperature = 30

if temperature > 25:
    print("It is hot")
```

Python also supports chained comparisons:

```
age = 20

if 18 <= age < 30:
    print("Young adult")
```

This is equivalent to:

```
if age >= 18 and age < 30:
    print("Young adult")
```

However, Python evaluates the middle expression only once in a chained comparison.

6. Logical Operators

Python provides three main logical operators:

and

Returns a truthy result only when both conditions are true.

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("Access allowed")
```

or

The result is truthy when at least one condition is true.

```
day = "Friday"

if day == "Friday" or day == "Saturday":
    print("Weekend")
```

not

Reverses the truth value.

```
logged_in = False

if not logged_in:
    print("Please log in")
```

Python's **and** and **or** operators are particularly important because they use **short-circuit evaluation**.

7. Membership Operators

Python provides:

- `in`
- `not in`

They test whether an element exists inside a collection.

```
fruits = ["apple", "banana", "orange"]

if "apple" in fruits:
    print("Apple exists")
```

Another example:

```
username = "admin"

if username not in ["guest", "visitor"]:
    print("Special user")
```

The `in` and `not in` operators can work with objects that support membership testing, such as sequences and sets.

8. Identity Operators

Python also provides:

- `is`
- `is not`

These check whether two references point to the **same object**, rather than simply containing equal values.

```
x = None

if x is None:
    print("No value")
```

A common best practice is:

```
if value is None:
```

rather than:

```
if value == None:
```

because `None` is a singleton object and `is` checks identity.

9. Truthy and Falsy Values

One of Python's most useful features is that conditions do not always have to explicitly produce `True` or `False`.

Python can determine whether an object is **truthy** or **falsy**.

Common falsy values

```
False
None
0
0.0
""
[]
()
{}
set()
```

Most other objects are considered truthy by default. Python also allows user-defined classes to control their truth value using `__bool__()` or `__len__()`.

Example:

```
name = ""

if name:
    print("Name exists")
else:
    print("Name is empty")
```

Output:

```
Name is empty
```

This is usually cleaner than:

```
if name != "":
```

10. Short-Circuit Evaluation

Short-circuit evaluation means Python may stop evaluating a logical expression before evaluating every part of it.

With and

```
x = 0

if x != 0 and 10 / x > 2:
    print("Valid")
```

Python evaluates:

```
x != 0
```

which is `False`.

Therefore, it does not evaluate:

```
10 / x
```

This prevents a division-by-zero error.

With `or`

```
name = "Salma"

if name == "Salma" or expensive_function():
    print("Found")
```

Because the first condition is true, Python does not need to evaluate the second condition.

This can improve both **performance and safety** when used appropriately. Python's documentation confirms that `and` and `or` evaluate their second operands only when necessary.

11. Nested Conditions

A conditional statement can contain another conditional statement.

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Access allowed")
    else:
        print("ID required")
else:
    print("Not old enough")
```

Nested conditions are useful when the second decision depends logically on the first.

However, excessive nesting makes code harder to read.

Less readable

```
if user:
    if user.active:
        if user.has_permission:
            print("Allowed")
```

Better

```
if user and user.active and user.has_permission:
    print("Allowed")
```

Or, when the conditions represent separate business rules, helper functions can make the code even clearer.

12. Conditional Expressions

Python supports a one-line conditional expression, sometimes called the **ternary operator**.

Syntax

```
value_if_true if condition else value_if_false
```

Example:

```
age = 20

status = "Adult" if age >= 18 else "Minor"

print(status)
```

Output:

```
Adult
```

This is useful for short, simple decisions.

Good use

```
maximum = a if a > b else b
```

Poor use

```
result = "A" if x > 90 else "B" if x > 80 else "C" if x > 70 else "D"
```

Although legal Python, it becomes difficult to read. A normal `if-elif-else` structure is better for complicated logic.

13. Comparison with Other Programming Languages

Python vs C++

Python:

```
if x > 10:
    print("Large")
else:
    print("Small")
```

C++:

```
if (x > 10) {
    std::cout << "Large";
} else {
    std::cout << "Small";
}
```

Main differences

- Python uses indentation.
 - C++ normally uses {}.
 - Python does not require parentheses around the condition.
 - C++ uses semicolons.
 - Python is dynamically typed.
 - C++ is statically typed.
-

Python vs Java

Python:

```
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

Java:

```
if (score >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

```
}
```

Java requires a Boolean expression inside parentheses and uses braces to define blocks.

Python vs C#

Python:

```
if temperature > 30:
    print("Hot")
```

C#:

```
if (temperature > 30)
{
    Console.WriteLine("Hot");
}
```

C# uses braces and explicit Boolean expressions, while Python uses indentation.

Python vs JavaScript

Python:

```
if age >= 18:
    print("Adult")
```

JavaScript:

```
if (age >= 18) {
    console.log("Adult");
}
```

JavaScript also supports truthy and falsy values, but its coercion rules differ from Python.

For example, JavaScript has values such as `null`, `undefined`, `NaN`, and `""` that are falsy. Python's truth-value system is different and is based on values such as `None`, zero, and empty collections.

Python vs Go

Python:

```
if x > 10:
    print("Large")
```

Go:

```
if x > 10 {
    fmt.Println("Large")
}
```

Go also supports a useful feature where a short statement can appear before the condition:

```
if x := getValue(); x > 10 {
    fmt.Println(x)
}
```

The Go specification explicitly supports this form.

Python vs Rust

Python:

```
if x > 10:
    print("Large")
```

Rust:

```
if x > 10 {
    println!("Large");
}
```

Rust uses braces and requires the condition to be Boolean. Rust is statically typed, while Python is dynamically typed.

14. Comparison Table

Feature	Python		C++	Java		C#	JavaScript	Go	Rust
if	Yes	Yes		Yes	Yes	Yes	Yes	Yes	Yes
elif	Yes	No		No	No	No	No	No	No

Feature	Python	C++	Java	C#	JavaScript	Go	Rust
else if	No, uses elif	Yes	Yes	Yes	Yes	else if	else if
Braces required	No	Usually	Usually	Usually	Usually	Yes	Yes
Indentation significant	Yes	No	No	No	No	No	No
Ternary expression	Yes	Yes	Yes	Yes	Yes	Limited syntax	Yes
Truthy/falsy objects	Yes	Limited/different	More restricted	More restricted	Yes	Boolean- focused	Boolean- focused
switch/match	match	switch	switch	switch	switch	switch	match

15. Readability

Python's conditional syntax is generally highly readable because it avoids unnecessary punctuation.

For example:

```
if temperature > 30:
    print("Hot")
elif temperature > 20:
    print("Warm")
else:
    print("Cold")
```

The structure is visually close to natural language.

However, readability depends on how the conditions are designed. Extremely long conditions can become difficult to understand:

```
if age >= 18 and has_id and account_active and not suspended and country in
allowed_countries:
    ...
```

In such situations, breaking the logic into meaningful variables or functions is better.

16. Performance

For a small number of conditions, performance differences are usually insignificant.

An `if-elif` chain generally checks conditions sequentially until one succeeds.

For example:

```
if x == 1:
    ...
elif x == 2:
    ...
elif x == 3:
    ...
```

If `x == 3`, Python has to reach the third comparison.

For large-scale dispatch problems, a dictionary or another appropriate data structure may be more efficient:

```
actions = {
    "start": start_game,
    "stop": stop_game,
    "pause": pause_game
}

actions["start"]()
```

However, **readability and correctness should normally be considered before attempting micro-optimizations.**

17. Best Practices for Python Conditional Statements

1. Keep conditions simple

Prefer:

```
if age >= 18:
```

instead of unnecessarily complicated expressions.

2. Use `elif` instead of deeply nested `if`

Prefer:

```
if score >= 90:
```

```
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C"
```

3. Use truthiness when appropriate

Prefer:

```
if items:
```

instead of:

```
if len(items) > 0:
```

4. Use `is None`

```
if result is None:
```

5. Use short-circuit evaluation intelligently

```
if user is not None and user.is_active:
    ...
```

6. Avoid extremely complicated ternary expressions

Use normal `if-elif-else` when the logic becomes difficult to read.

7. Avoid excessive nesting

Extract complicated conditions into functions when necessary.

18. Conclusion of Task 1

Python's `if`, `elif`, and `else` statements provide a simple and flexible mechanism for decision-making. Python's use of indentation, truth-value testing, comparison operators, logical operators, membership operators, and short-circuit evaluation allows developers to write concise and readable conditional logic.

Compared with languages such as C++, Java, C#, JavaScript, Go, and Rust, Python generally uses less syntactic punctuation and places greater emphasis on readability.

For effective Python programming, developers should prefer **simple conditions, meaningful variable names, appropriate use of truthy/falsy values, limited nesting, and clear separation of complex logic**. Performance optimizations should be considered only when they solve an actual performance problem.

Task 2: Switch Case in Python

1. Introduction

A `switch-case` statement is a control-flow structure used by many programming languages to select one action from several alternatives based on the value of an expression.

For example, a language might provide:

```
switch value
  case 1:
    ...
  case 2:
    ...
  default:
    ...
```

Languages such as C++, Java, C#, JavaScript, and Go provide `switch`, while Rust provides the closely related `match` construct.

Historically, Python did **not** have a traditional `switch-case` statement.

2. Why Python Traditionally Did Not Have Switch-Case

Before Python 3.10, developers had to use other techniques.

The main reason was not simply that Python's designers "forgot" to add `switch-case`. There were discussions and formal proposals for a `switch` statement.

For example, **PEP 3103**, titled *A Switch/Case Statement*, was formally rejected. The proposal noted that `if/elif` chains already handled the common use case, while a new construct would need to justify its additional syntax and semantics.

The proposal specifically observed that the main motivations for a `switch` were:

1. Avoid repeating the variable in every comparison.
2. Potentially improve efficiency.

However, these benefits were considered relatively modest for Python's use cases.

Python's general design philosophy strongly favors:

- Readability
- Simplicity
- Explicit behavior
- Avoiding unnecessary language complexity

Instead of adding a traditional C-style switch, Python historically provided several flexible alternatives.

3. Method 1: `if-elif-else`

The most straightforward solution is:

```
command = "start"

if command == "start":
    print("Starting...")
elif command == "stop":
    print("Stopping...")
elif command == "pause":
    print("Pausing...")
else:
    print("Unknown command")
```

Python's documentation explicitly describes an `if/elif` sequence as a substitute for the `switch` or `case` statements found in other languages.

Advantages

- Simple
- Flexible
- Easy to understand
- Works with arbitrary conditions
- No special data structure required

Disadvantages

- Long chains can become repetitive.
- Conditions are normally evaluated sequentially.

- Large dispatch tables may be harder to maintain.
-

4. Method 2: Dictionary-Based Dispatch

Python dictionaries can map values to results or actions.

Example:

```
def start():
    print("Starting")

def stop():
    print("Stopping")

def pause():
    print("Pausing")

actions = {
    "start": start,
    "stop": stop,
    "pause": pause
}

command = "start"

if command in actions:
    actions[command]()
else:
    print("Unknown command")
```

The dictionary acts as a lookup table.

Advantages

- Very concise
- Excellent for many simple value-to-function mappings
- Easy to extend
- Can provide fast average lookup for hashable keys

Disadvantages

- More complicated conditions are difficult to express.
 - The mapping must be constructed and maintained.
 - It may be less obvious to beginners.
-

5. Method 3: Function Mapping

Function mapping is closely related to dictionary dispatch.

```
def add():
    return 10 + 5

def subtract():
    return 10 - 5

operations = {
    "add": add,
    "subtract": subtract
}

operation = "add"

result = operations[operation]()
print(result)
```

This approach is especially useful when each case performs a different operation.

6. Method 4: Dictionary Mapping to Values

A dictionary does not have to contain functions.

```
days = {
    1: "Monday",
    2: "Tuesday",
    3: "Wednesday",
    4: "Thursday",
    5: "Friday"
}

day = 3

print(days.get(day, "Invalid day"))
```

Output:

Wednesday

Here, `.get()` provides a convenient default value.

7. Method 5: Object-Oriented Polymorphism

Another way to avoid large conditional structures is **polymorphism**.

Instead of asking:

```
if animal == "dog":  
    ...  
elif animal == "cat":  
    ...
```

different objects can implement their own behavior.

```
class Dog:  
    def speak(self):  
        return "Woof"  
  
class Cat:  
    def speak(self):  
        return "Meow"  
  
animals = [Dog(), Cat()]  
  
for animal in animals:  
    print(animal.speak())
```

The program does not need a large conditional statement because each object knows how to perform its own behavior.

Advantages

- Excellent for large object-oriented systems
- Improves maintainability
- Makes behavior easier to extend

Disadvantages

- More code for simple problems
- Can be unnecessary for a small number of cases

8. Other Common Patterns Before Python 3.10

Python programmers also used:

Nested `if` statements

```
if command == "admin":
    if action == "delete":
        print("Delete")
```

Tuple/list membership

```
if day in ("Saturday", "Sunday"):
    print("Weekend")
```

Dictionary lookup

```
result = {
    "A": 90,
    "B": 80,
    "C": 70
}.get(grade, 0)
```

Functions as first-class objects

```
def hello():
    print("Hello")

def goodbye():
    print("Goodbye")

actions = {
    "hello": hello,
    "goodbye": goodbye
}

actions["hello"]()
```

These approaches demonstrate an important Python feature: the language does not always need a dedicated control-flow keyword when existing data structures and functions can solve the problem cleanly.

9. Structural Pattern Matching in Python

3.10

Python 3.10 introduced **Structural Pattern Matching** through the `match` and `case` keywords.

This was a major addition to Python's conditional capabilities.

It looks similar to switch-case:

```
command = "start"

match command:
    case "start":
        print("Starting")
    case "stop":
        print("Stopping")
    case "pause":
        print("Pausing")
    case _:
        print("Unknown command")
```

The `_` pattern works as a wildcard and can serve as a general fallback.

Structural pattern matching is specified by PEP 634, with PEP 636 providing a tutorial and PEP 635 explaining the motivation and design decisions.

10. Basic match-case Syntax

General form

```
match expression:
    case pattern1:
        statement
    case pattern2:
        statement
    case _:
        statement
```

Example:

```
status = 404

match status:
    case 200:
        print("OK")
    case 404:
        print("Not Found")
    case 500:
        print("Server Error")
    case _:
        print("Unknown Status")
```

Output:

```
Not Found
```

11. Matching Multiple Values

Python can combine alternatives using the `|` pattern.

```
day = "Saturday"

match day:
    case "Saturday" | "Sunday":
        print("Weekend")
    case _:
        print("Weekday")
```

This is more expressive than a traditional simple value comparison.

12. Sequence Patterns

Pattern matching can examine the structure of sequences.

```
point = (10, 20)

match point:
    case (0, 0):
        print("Origin")
    case (x, 0):
        print("On X-axis")
    case (0, y):
        print("On Y-axis")
    case (x, y):
        print("Point:", x, y)
```

This is one of the major differences between Python's `match-case` and a traditional switch statement.

The program is not merely asking:

"Does this value equal 10?"

It can ask:

"Does this object have this particular structure?"

13. Mapping Patterns

Python can match dictionary-like structures.

```
user = {
    "name": "Ali",
    "role": "admin"
}

match user:
    case {"role": "admin"}:
        print("Administrator")
    case {"role": "user"}:
        print("Regular user")
    case _:
        print("Unknown role")
```

This is called a **mapping pattern**.

14. Class Patterns

Pattern matching can also work with objects.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

point = Point(0, 10)

match point:
    case Point(0, y):
        print("On Y-axis")
    case Point(x, 0):
        print("On X-axis")
    case Point(x, y):
        print("Other point")
```

Python's structural pattern matching specification includes literal, capture, wildcard, value, group, sequence, mapping, and class patterns.

15. Pattern Guards

A case can have an additional condition using `if`.

```
number = 10
```

```
match number:
    case x if x > 0:
        print("Positive")
    case x if x < 0:
        print("Negative")
    case _:
        print("Zero")
```

The guard provides extra flexibility beyond simple value matching.

16. Important Difference: `match` Is Not Just Switch

It is tempting to say:

"Python finally added switch-case in Python 3.10."

This is convenient for beginners, but technically incomplete.

Python added **structural pattern matching**, not merely a traditional C-style switch.

Traditional switch:

value → compare with cases → execute matching case

Python `match`:

value → test patterns → possibly extract data → optionally evaluate guard → execute case

Therefore, `match-case` is significantly more powerful than a simple switch.

17. Comparison with C++ Switch

C++:

```
switch (number) {
    case 1:
        std::cout << "One";
        break;

    case 2:
```



```
        std::cout << "Two";  
        break;  
  
    default:  
        std::cout << "Other";  
}
```

C++ switch statements select execution based on the value of a condition, and traditional C++ switch behavior allows execution to fall through from one case to another unless control is explicitly stopped.

Python:

```
match number:  
    case 1:  
        print("One")  
    case 2:  
        print("Two")  
    case _:  
        print("Other")
```

Python does not use C-style fall-through between cases.

18. Comparison with Java

Java supports both traditional switch statements and modern switch expressions/patterns.

Example:

```
switch (day) {  
    case 1 -> System.out.println("Monday");  
    case 2 -> System.out.println("Tuesday");  
    default -> System.out.println("Other");  
}
```

Modern Java has significantly expanded switch functionality, including patterns and guards. The current Java Language Specification documents switch statements with case constants, patterns, guards, and exhaustiveness rules.

Python's `match` is conceptually closer to modern pattern matching than to the older C-style switch.

19. Comparison with C#

C# supports traditional `switch` statements as well as modern pattern-based switch expressions.

Example:

```
string result = number switch
{
    1 => "One",
    2 => "Two",
    _ => "Other"
};
```

C# switch expressions can evaluate an expression based on a list of candidate patterns.

Python `match` has a similar pattern-oriented philosophy:

```
result = None

match number:
    case 1:
        result = "One"
    case 2:
        result = "Two"
    case _:
        result = "Other"
```

20. Comparison with JavaScript

JavaScript provides the traditional `switch` statement.

```
switch (day) {
    case "Monday":
        console.log("Start");
        break;

    case "Friday":
        console.log("Almost weekend");
        break;

    default:
        console.log("Normal day");
}
```

JavaScript's `switch` compares the switch expression against case expressions using strict equality. If `break` is omitted, execution can fall through into subsequent cases.

Python's `match` avoids this traditional fall-through behavior.

21. Comparison with Go

Go has a built-in `switch`.

```
switch day {
case "Saturday":
    fmt.Println("Weekend")
case "Sunday":
    fmt.Println("Weekend")
default:
    fmt.Println("Weekday")
}
```

One interesting difference is that Go does **not** require a `break` after every case. Go's switch cases normally finish automatically. Go supports both expression switches and type switches.

22. Comparison with Rust

Rust uses `match` rather than `switch`.

```
match number {
  1 => println!("One"),
  2 => println!("Two"),
  _ => println!("Other"),
}
```

Rust's `match` is very powerful and is based on patterns. Rust requires the match to cover all possible cases, commonly using `_` as a catch-all. The Rust Reference describes `match` as branching on patterns and also supports pattern guards and alternatives.

Python's `match` was influenced by the broader concept of pattern matching found in languages such as Rust, Haskell, Scala, and others.

23. Comparison Table

[illegible]

Feature	Python if	Python Dictionary	Python match	C++ switch	Java switch	C# switch	JS switch	Go switch	Rust match
Complex conditions	Excellent	Limited	Good with guards	Limited	Good	Excellent	Limited	Good	Excellent
Pattern matching	No	No	Yes	Limited	Yes	Yes	No	Type switch	Yes
Data extraction	No	No	Yes	No	Yes	Yes	No	Limited	Yes
Fall-through	No	No	No	Yes	Possible	No normal fall-through	Yes	No	No
Function dispatch	Possible	Excellent	Possible	Possible	Possible	Possible	Possible	Possible	Possible
Readability	High	High for mappings	High for structured data	High for simple cases	High	High	High	High	High
Flexibility	High	Medium	Very high	Medium	High	Very high	Medium	High	Very high

24. Readability Analysis

if-elif-else

Best for:

```
if temperature > 30:
    ...
elif temperature > 20:
    ...
else:
    ...
```

This is easy to understand because the conditions themselves are visible.

Dictionary dispatch

Best for:

```
actions = {
    "start": start,
```

```
"stop": stop,  
"pause": pause  
}
```

This is extremely readable when the problem is simply:

"Given this key, perform this function."

match-case

Best for structured data:

```
match point:  
    case (0, 0):  
        ...  
    case (x, 0):  
        ...  
    case (0, y):  
        ...
```

Here, `match` communicates the structure of the data directly.

25. Performance Analysis

Performance depends heavily on the specific problem and implementation.

if-elif

Conditions are evaluated sequentially.

For example:

```
if x == 1:  
    ...  
elif x == 2:  
    ...  
elif x == 3:  
    ...
```

A late match requires more comparisons.

Dictionary dispatch

A dictionary can provide efficient average-case lookup for hashable keys:

```
actions[x]()
```

This can be attractive when there are many simple, fixed mappings.

match-case

`match` is designed for pattern matching, so its implementation is more complex than a simple chain of equality tests. The exact performance depends on the patterns, objects involved, and Python implementation.

Therefore, it is not correct to claim that `match` is **always faster** than `if-elif`, or that dictionaries are **always faster**. Performance should be measured for the actual application.

For ordinary application code, readability and maintainability usually matter more than tiny differences in branch performance.

26. Maintainability Analysis

if-elif

Advantages

- Very easy to understand
- Flexible
- Works with arbitrary expressions

Disadvantages

- Long chains become repetitive
 - Can become difficult to maintain
-

Dictionary Dispatch

Advantages

- Easy to add new actions
- Excellent separation between keys and functions
- Compact

Disadvantages

- Less suitable for complicated conditions
- Can become confusing if functions have inconsistent behavior

match-case

Advantages

- Excellent for structured data
- Supports pattern matching
- Supports data extraction
- Supports guards
- Often cleaner than deeply nested conditions

Disadvantages

- Requires Python 3.10 or newer
 - Has a larger learning curve
 - Can be unnecessarily complex for simple value comparisons
-

27. Real-World Use Cases

Use Case 1: Menu System

Dictionary dispatch works well:

```
def add():
    print("Adding")

def delete():
    print("Deleting")

def update():
    print("Updating")

actions = {
    "add": add,
    "delete": delete,
    "update": update
}

command = "add"

actions.get(command, lambda: print("Unknown command"))()
```

Use Case 2: Grade Classification

Use if-elif:

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "D"
```

This is better than `match` because the conditions are ranges rather than exact structural patterns.

Use Case 3: Network Packet Processing

`match` can be useful when the data has a known structure.

```
packet = ("TCP", 80)

match packet:
    case ("TCP", 80):
        print("HTTP traffic")
    case ("TCP", 443):
        print("HTTPS traffic")
    case ("UDP", port):
        print("UDP port:", port)
    case _:
        print("Unknown packet")
```

Use Case 4: API Response Processing

```
response = {
    "status": 200,
    "data": "OK"
}

match response:
    case {"status": 200, "data": data}:
        print("Success:", data)

    case {"status": 404}:
        print("Not found")

    case {"status": status}:
        print("Other status:", status)

    case _:
```



```
print("Invalid response")
```

This demonstrates why `match` can be much more powerful than traditional switch-case.

28. Choosing the Right Technique

A useful decision rule is:

Use `if-elif-else` when:

- Conditions involve ranges.
- Conditions involve different variables.
- Conditions involve complex Boolean logic.
- You need maximum flexibility.

Example:

```
if age >= 18 and has_id:
    ...
elif age >= 13:
    ...
else:
    ...
```

Use dictionary dispatch when:

- You are mapping keys directly to functions.
- You have many simple commands.
- Each key has an independent action.

Example:

```
actions = {
    "start": start,
    "stop": stop,
    "pause": pause
}
```

Use `match-case` when:

- You are using Python 3.10+.
- You are matching specific values.
- You are working with tuples, dictionaries, classes, or structured data.

- You need pattern matching or data extraction.

Example:

```
match command:
    case ("move", direction):
        move(direction)
    case ("stop",):
        stop()
    case _:
        unknown_command()
```

29. Overall Comparison

Situation	Recommended Approach
Simple condition	if
Several different conditions	if-elif-else
Range checking	if-elif-else
Simple value-to-function mapping	Dictionary
Many command handlers	Dictionary dispatch
Structured data	match-case
Tuple/list patterns	match-case
Dictionary/object patterns	match-case
Complex Boolean conditions	if-elif-else
Object-specific behavior	Polymorphism
Very simple exact values	match-case or dictionary

30. Best Practices for `match-case`

1. Use `_` as a fallback when appropriate

```
match command:
    case "start":
        ...
    case "stop":
        ...
    case _:
        ...
```

2. Keep patterns readable

Avoid creating extremely complicated patterns just because Python allows them.

3. Use guards when they clarify the logic

```
match number:
    case x if x > 100:
        print("Large")
```

4. Don't use `match` when a simple `if` is clearer

For example:

```
if age >= 18:
    print("Adult")
```

is much clearer than trying to force the problem into pattern matching.

5. Use dictionary dispatch for command tables

If the problem is simply:

```
key → function
```

a dictionary is often the cleanest solution.

31. Final Conclusion of Task 2

Python historically did not provide a traditional `switch-case` statement because its designers considered existing mechanisms such as `if-elif-else`, dictionaries, functions, and object-oriented techniques sufficient for many use cases. Formal switch proposals were considered but rejected, partly because the benefits did not clearly justify introducing another specialized construct into the language.

Python 3.10 changed the situation by introducing **Structural Pattern Matching** through `match-case`. Unlike a traditional switch statement, `match-case` is not limited to comparing a single value against constants. It can match literals, sequences, mappings, classes, alternatives, and other patterns, while also allowing guards and extracting values.

Therefore, Python now provides several powerful approaches:

- **`if-elif-else`** for general conditions.
- **Dictionary dispatch** for simple key-to-action mappings.
- **Polymorphism** for object-oriented designs.
- **`match-case`** for structured and pattern-based data.